

DATE : 01.10.2026
DT/NT : DT
LESSON : LINUX
SUBJECT: SHELL SCRIPTING BASIC
SESSION : 8
BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



Shell Scripting



BASH

THE BOURNE-AGAIN SHELL



Table of Contents



- ▶ Review
 - Shell
 - Bash
- ▶ Bash Prompt & PS1
- ▶ Shell Scripts
- ▶ Script Anatomy & Shebang
- ▶ Variables & Environment Variables
- ▶ User Input with read
- ▶ Quoting & Command Substitution
- ▶ Positional & Special Parameters
- ▶ Arithmetic
- ▶ Script Debugging
- ▶ Exercises



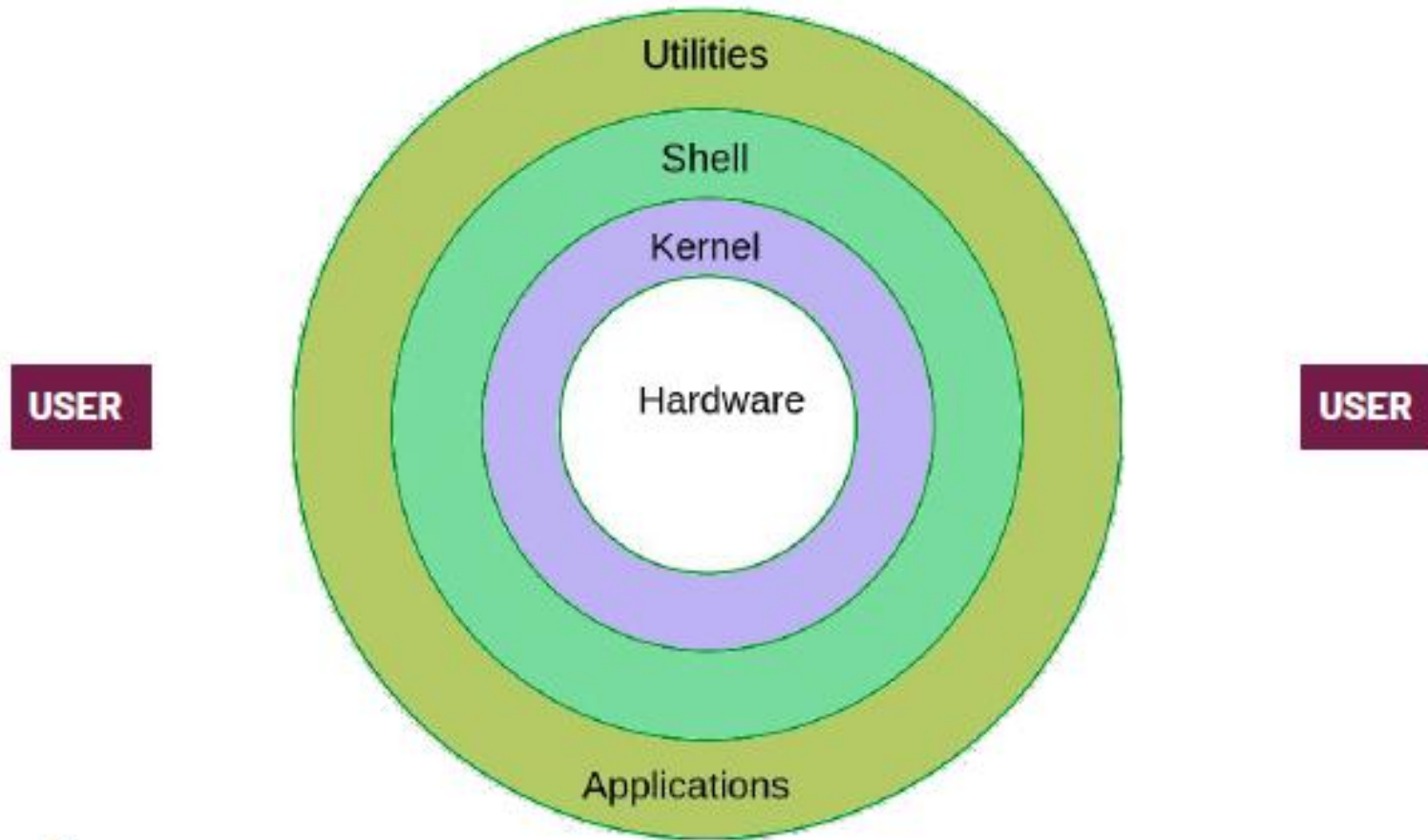
Shell

- A command interpreter that provides an interface between the user and the operating system.

Bash

- Bash (Bourne Again SHell) is one shell and also provides a scripting language.
- Other shells include Zsh, Fish, and KornShell.

Components of Linux



What is SHELL?

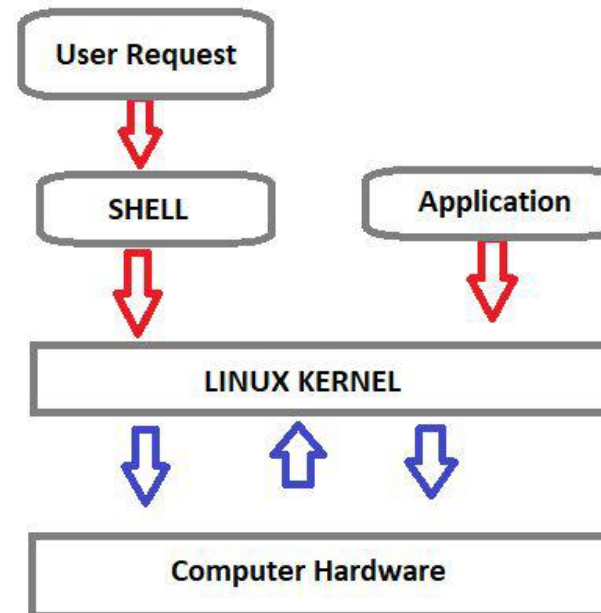


➤ What is a Shell?

A shell interprets commands, runs programs, and returns their output. It provides a command-line interface and can also execute shell scripts.

Important:

Linux is the operating system; Bash is one possible shell used on Linux.



SHELL



1

Bash : Bourne Again shell

The standard GNU shell, intuitive and flexible

2

ksh : Korn shell

A superset of the Bourne shell

3

csh : C shell

The syntax of this shell resembles that of the C programming language

4

tcsh: TENEX C shell

A Superset of the common C shell, enhancing user-friendliness and speed

5

zsh : Z Shell

An extended Bourne shell with a large number of improvements, including some features of Bash, ksh, and tcsh.

Bash Prompt

LINUX@BASH-PROMPT:~\$ 	
EXAMPLE OF BASH PROMPT	PS1=' \u@\h:\w\\$ '
PS1 - shell prompt variable	\u - the username of the current user
\h - the hostname up to the first '.'	\w - the current working directory
\# - if the effective UID is 0 (root / superuser), a #, otherwise a \$	
LINUXCONFIG.ORG	



The Bash prompt



➤ Bash Prompt

The prompt indicates that the shell is ready to receive the next command.

PS1

PS1 is Bash's primary interactive prompt string.

Common prompt escapes:

\u → username \h → hostname

\w → current directory \\$ → \$ or #

```
user@host:~$
```

The Bash prompt



Changing PS1

You can customize the interactive Bash prompt by changing PS1.

Example:

```
PS1='\u@\h:\w$ '
```

Check the current value:

```
echo "$PS1"
```

```
linuxuser@techproeducation: ~$  
linuxuser@techproeducation:~$ PS1="\[\e[1;32m\]\u@\h:\[\e[1;34m\]\w\[\e[m\]\$ "  
linuxuser@techproeducation:~$ PS1='\u@\h \[\e[1;34m\]\w\[\e[m\] 🚀 \$ '  
linuxuser@techproeducation ~ 🚀 $ PS1='[\t] \u@\h:\w$ '  
[22:12:18] linuxuser@techproeducation:~$ PS1='\u@\h:\w$(__git_ps1 " (%s)")\  
$ '  
linuxuser@techproeducation:~$ PS1="\[\e[1;32m\]\u@\h:\[\e[1;34m\]\w\[\e[m\]\$ "  
linuxuser@techproeducation:~$ |
```

The Bash prompt



Bash prompt can be customized by using special characters. Here is the most used characters and their meaning.

Special Character	Explanation
\d	The date in "Weekday Month Date" format (e.g., "Sun Apr 12")
\h	the hostname up to the first `.`
\H	the hostname
\s	the name of the shell, the basename of \$0 (the portion following the final slash)
\t	the current time in 24-hour HH:MM:SS format
\T	the current time in 12-hour HH:MM:SS format
\@	the current time in 12-hour am/pm format
\u	the username of the current user

The Bash prompt



Bash prompt can be customized by using special characters. Here is the most used characters and their meaning.

Special Character	Explanation
<code>\v</code>	the version of bash (e.g., 2.00)
<code>\V</code>	the release of bash, version + patch level (e.g., 2.00.0)
<code>\w</code>	the current working directory
<code>\W</code>	the basename of the current working directory
<code>!\</code>	the history number of this command

The Bash prompt



Make PS1 permanent

For interactive Bash sessions, place your PS1 assignment in ~/.bashrc.

Example:

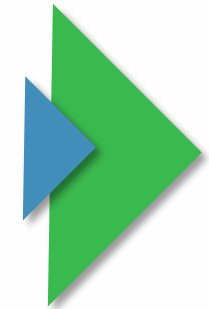
```
PS1='\u@\h:\w\$ '
```

Reload without logging out:

```
source ~/.bashrc
```

Tip:

Login-shell startup files may source ~/.bashrc; do not assume every shell uses the same startup file.



Shell Scripts



Shell Scripts



What is Shell Scripting?

Shell scripting is the practice of writing a sequence of shell commands in a text file so they can be executed automatically.

A shell script is a text file containing shell commands.

Common uses:

File processing • automation • system administration • deployment tasks

Shell Scripts



Why Use Shell Scripts?

Long or repetitive command sequences can be combined into one reusable script.

A script can be saved, reviewed, version-controlled, and executed again.

AWS/DevOps examples:

Log collection • deployment checks • backups • server configuration • health checks

Shell Script Anatomy



1. Shebang

```
#!/usr/bin/env bash
```

2. Comments

```
# This script checks the server
```

3. Commands

```
echo "Hello AWS-DevOps"
```

4. Variables / logic

```
HOSTNAME=$(hostname)
```

```
echo "Server: $HOSTNAME"
```

Quoting in Bash



Single quotes: literal text

`name="TechPro" → echo '$name' → $name`

Double quotes: variable expansion

`echo "$name" → TechPro`

Best practice

Use double quotes around variables when you want their value preserved as one argument.

Example:

`echo "Deploying to $ENVIRONMENT"`

Command Substitution



Run a command and store its output:

```
HOST=$(hostname)
```

```
DATE=$(date)
```

Use the result later:

```
echo "Server: $HOST"
```

```
echo "Deployment started: $DATE"
```

Preferred modern syntax:

Use `$(command)` rather than backticks as the primary syntax.

Debugging a Bash Script



Trace a script from the command line:

```
bash -x script.sh
```

Trace inside a script:

```
set -x
```

```
set +x
```

Why?

Bash prints commands after expansion and before execution, making it easier to find where a script behaves unexpectedly.

Exercise 1



1. Exercise 1 — First AWS/DevOps Script

Create:

system-info.sh

The script must display:

User:

Hostname:

Current Directory:

Shell:

Kernel:

Make it executable and run it.

Use commands already learned in Days 1–7.

Shell Scripts



Run Your Own Scripts Without ./

Create a personal commands directory:

```
mkdir -p "$HOME/bin"
```

Add it to PATH for the current session:

```
export PATH="$HOME/bin:$PATH"
```

Persist it in ~/.bashrc:

```
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc
```

```
source ~/.bashrc
```

Do not add '.' to PATH.

Variables



- **What is a Shell Variable?**

A shell variable is a named value stored by the shell.

Naming rules:

Use letters, digits, and underscores; the name must not start with a digit.

Examples:

```
PROJECT_NAME=aws-devops
```

```
server_count=3
```


Shell Variables vs Environment Variables



Shell variable

`APP_ENV=production`

Available in the current shell.

Environment variable

`export APP_ENV`

Exported variables are inherited by child processes.

Remove a variable:

`unset APP_ENV`

Variables



Variable Assignment

There must be no spaces around =.

Correct:

```
samplendir="/etc"  
echo "$samplendir"
```

Incorrect:

```
samplendir = /etc
```

Use \$ when reading/expanding the variable, not when assigning it.

```
$ myvar='Hello World'  
$ echo $myvar  
Hello World  
$ newvar="More $myvar"  
$ echo $newvar  
More Hello World  
$ newvar='More $myvar'  
$ echo $newvar  
More $myvar
```

Console input



Console Input with read

Basic syntax:

```
read -r name
```

Better user experience:

```
read -r -p "Enter your name: "  
name
```

```
echo "Hello, $name"
```

-r

Prevents backslashes from being treated as escape characters while reading input.

```
#!/bin/bash
```

```
echo "Enter your name: "
```

```
read name
```

```
echo Hello $name
```

```
~
```

```
$
```

```
[ec2-user@ip-172-31-36-108 ~]$ ./run.sh
```

```
Enter your name:
```

```
Raymond
```

```
Hello Raymond
```

```
[ec2-user@ip-172-31-36-108 ~]$
```

Console input



Reading Multiple Values

```
read -r -p "Enter first name: " first
read -r -p "Enter last name: " last
echo "Name: $first $last"
```

For sensitive input:

```
read -r -s -p "Password: " password
```

Use -s when you do not want typed input displayed on the terminal.

```
#!/bin/bash
```

```
read -p "Enter Your Name: " username
echo "Welcome $username!"
```

```
#!/bin/bash
```

```
read -s -p "Enter Password: " pswd
echo $pswd
```

```
#!/bin/bash
```

```
read -sp "Enter Password: " pswd
echo $pswd
```

```
#!/bin/bash
```

```
echo What cars do you like?
```

```
read car1 car2 car3
```

```
echo Your first car was: $car1
echo Your second car was: $car2
echo Your third car was: $car3
```



Positional Parameters

- `$0` → script name
- `$1, $2, $3 ...` → positional arguments
- `$#` → number of positional arguments
- `"$@"` → all positional arguments, preserving argument boundaries
- **Special Parameters**
- `$?` → exit status of the last command
- `$$` → PID of the current shell/script
- **Note:**
- `$USER` and `$HOSTNAME` are shell variables, not command-line arguments.

Positional Parameters — Example



Script:

```
echo "Script: $0"  
echo "First argument: $1"  
echo "Second argument: $2"  
echo "Argument count: $#"  
printf "Argument: %s\n" "$@"
```

Run:

```
./info.sh "AWS DevOps" Linux
```

For arguments beyond \$9, use braces:

```
${10}
```

Simple Arithmetic



Arithmetic in Bash

Preferred Bash syntax:

```
sum=$((num1 + num2))
```

```
((sum++))
```

Common operators:

+ - * / % ++ --

Modern Bash scripts should prefer native arithmetic over external expr.

Arithmetic Expressions



Arithmetic Expansion: `$(( ))`

Use it when you need an arithmetic result.

Examples:

```
sum=$((10 + 5))
```

```
total=$((servers * cost))
```

```
remainder=$((10 % 3))
```

Important:

Bash arithmetic uses integers; `10 / 3` produces 3.

```
#!/bin/bash
first_number=8
second_number=2

echo "SUM="`expr $first_number + $second_number`
echo "SUB="`expr $first_number - $second_number`
echo "MUL="`expr $first_number \* $second_number`
echo "DIV="`expr $first_number / $second_number`
```

```
$ chmod +x cal.sh
$ ./cal.sh
SUM=10
SUB=6
MUL=16
DIV=4
```


Arithmetic Expressions



Alternative / Older Arithmetic Syntax

expr:

expr 10 + 5

let:

let sum=num1+num2

Teaching note:

These forms still work, but for new Bash scripts prefer `$((...))` and `((...))`.

```
#!/bin/bash

number1=8
number2=2

let total=number1+number2
let diff=number1-number2
let mult=number1*number2
let div=number1/number2

echo "Total = $total"
echo "Difference = $diff"
echo "Multiplication = $mult"
echo "Division = $div"
```

```
$ ./run.sh
Total = 10
Difference = 6
Multiplication = 16
Division = 4
```

Arithmetic Expressions



Increment, Decrement & Modulo

- `((num++))` → increment after using the value
- `((++num))` → increment before using the value
- `((num--))` → decrement
- `((--num))` → decrement before using the value

Modulo:

`((remainder = 10 % 3))` → 1

Use these inside Bash arithmetic contexts.

```
#!/bin/bash
number=10
let new_number=number++
echo "Number = $number"
echo "New number = $new_number"

number=10
let new_number=--number
echo "Number = $number"
echo "New number = $new_number"
```

```
[ec2-user@ip-172-31-91-206 ~]$ ./run.sh
Number = 11
New number = 10
Number = 9
New number = 9
[ec2-user@ip-172-31-91-206 ~]$
```

Arithmetic Expressions



`$(( ))` vs `(( ))`

`$(( expression ))`

Produces an arithmetic value that can be assigned or printed.

```
result=$((num1 + num2))
```

`(( expression ))`

Evaluates an arithmetic expression as a Bash command.

```
((num++))
```

Both are native Bash features.

```
#!/bin/bash
```

```
number1=8
```

```
number2=2
```

```
echo "Total = $((number1+number2))"
```

```
((total=number1+number2))
```

```
echo "Total = $total"
```

```
█
```

```
~
```

```
[ec2-user@ip-172-31-91-206 ~]$ ./run.sh
```

```
Total = 10
```

```
Total = 10
```

```
[ec2-user@ip-172-31-91-206 ~]$ █
```

Exercise 2



1. Exercise 2 — Deployment Calculator

Create:

deployment-cost.sh

Run it like:

```
./deployment-cost.sh 5 20
```

Meaning:

\$1 = number of servers

\$2 = cost per server

Calculate and display:

Servers: 5

Cost per server: 20

Total: 100

Use positional parameters and Bash arithmetic.

Exercise 3



1. Exercise 3 — System Report

Create:

system-report.sh

The script must:

1. Ask for the student's name.
2. Capture the hostname with `$(hostname)`.
3. Ask for the number of servers.
4. Calculate double the server count.
5. Display a formatted report.

Use `read -r -p`, variables, command substitution, and `$(( ))`.

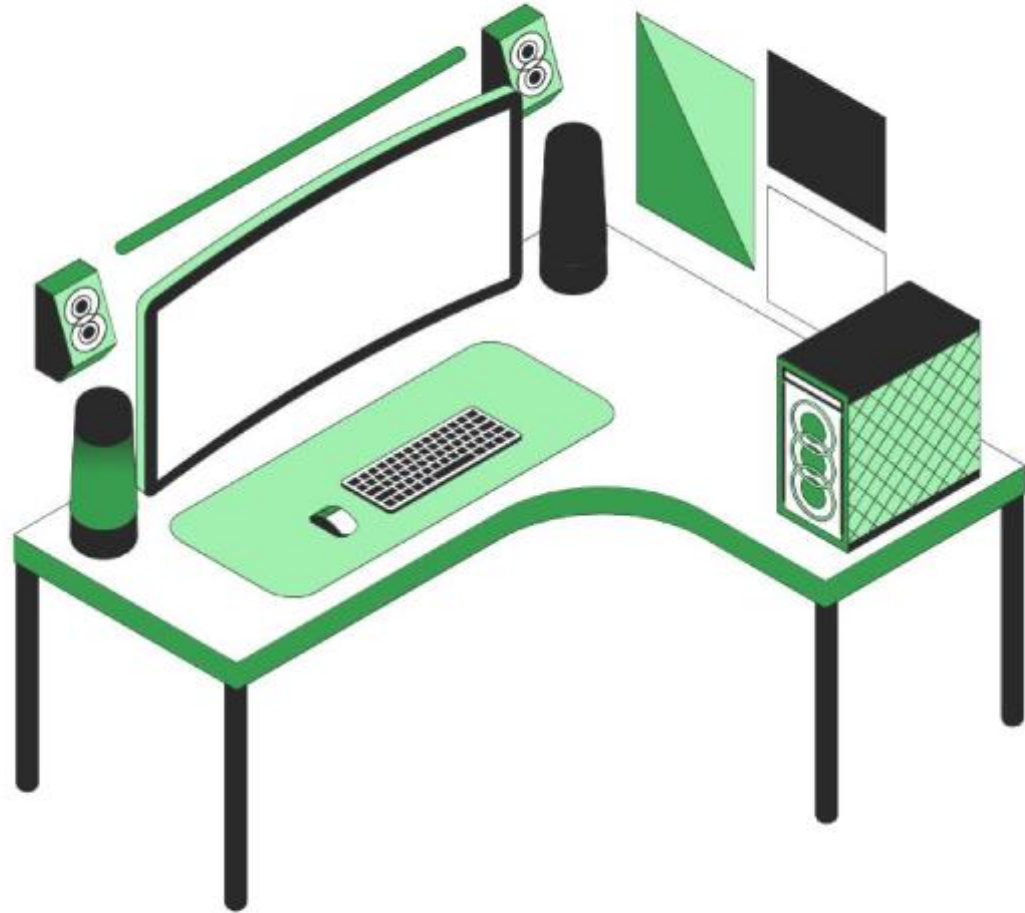
Exercise 4 — Bash Debugging Challenge

Create a script named deploy-check.sh

```
HOST=$(hostname)
echo "Checking $HOST"
read -r -p "Enter server count: " count
total=$((count * 2))
echo "Planned instances: $total"
```

Then:

1. Run the script normally.
2. Run it again with: `bash -x deploy-check.sh`
3. Identify each command Bash traces.
4. Add `set -x` / `set +x` around a section and compare the output.



Do you have any questions?

Send it to us! We hope you learned something new.